PAGE No.
DATE / /

24/6/26

Python Programming
Assignment - 7

1) What is the difference between local variables and global variables?

Ans:- Local variable : These are created inside a specific function. They have local scope. They are strictly private to a function. The moment the function finishes its execution, Python destroys the local variables to free up memory. They can be accessed within that same function where it is been declared.

- Global variable : These are declared at the top level of the program. outside the functions. They have a global scope, meaning function inside that file can look at them. They stay alive in memory for the entire duration of the program.

Basically, global variable can be accessible by anyone in the program whereas local variable cannot be accessible outside it function in the program.

2) Why should excessive use of global variables be avoided in large programs?

Ans: Excessive use of global variables be avoided in large programs because,

- Bug fixing : Because every function can see and change a global variable, one bad function can mess it up for everyone else. It is very hard to figure out who broke it.

- Reusing Code: If the function relies on a global variable, we cannot easily reuse that function for other inputs or tasks, we have rewrite the code entirely.

- Nameing Confusion: In large programs, it is very easy to accidentally give a new local variable the exact same name as a global one, which confuses the programmer and interpreter gives error which not allowed.

3) Explain the use of global keyword. When should it be used?

Ans: In Python, you can read a global variable from inside a function automatically. However, the when you try to reassign it (eg. x=5), Python assumes you are trying to create a brand new local variable named x.

The global keyword acts as an explicit overide to the Python interpreter: "Do not create a new local variable; bind this identifier to the one that already exists in global scope."

When to use it: You should only use it when a function must mutate a top-level state variable ~~(eg.,~~

4) What is the special variable __name__ in Python? Who assigns its value?

Ans: It is a built-in "dunder" (~~dauble~~ double-underscore) variable that holds the string name of the module currently being evaluated.

The Python Interpreter automatically assigns its value behind the scenes the millisecond a file is loaded into memory.

5) What is the meaning of __no__main__ in Python execution?

Ans: In Python, __main__ represents the name of top-level environment where your main program runs. It tells the Python interpreter which file is the starting point of execution.

Whenever Python runs a script, it automatically creates a special background variable named __name__. The value assigned to __name__ depends entirely on how you run the file.

6) Can a function return another function? Explain conceptually.

Ans: Yes, a function can absolutely return another function in Python. This is possible because functions in Python first-class objects.

To understand this conceptually, think of a function not just as a set of instructions, but as a regular peice of data just like string, integer or a list.

- Since you can pass a number into function and get a string back, you can also pass data into a function and get a recipe (a function) back.
- When you return a function, you are not returning the result of the function; you are returning the unexecuted function package itself.

7) What is module in Python? How is it different from a function?

Ans: A module in Python is simply a file containing Python code - variables, functions, classes - that you can reuse in other Python Programs. File with .py extension is treated as a module.

PAGE No.
DATE / /

Use of Modules-

- Reusability : write once, use anywhere
- Organisation : Keep code modular and manageable
- Avoid code Duplication : common code can be placed in a module.
- Easier Maintenance & Debugging.

In Python, a module is a file containing code, whereas a function is a specific block of code written inside a program or module to perform a single task.

8) Explain what happens to --name--:

- When a Python file is executed directly
- When the same file is imported as a module.

Ans : i) When a Python file is executed directly

When you run the program directly from your terminal, the Python interpreter automatically assign the special built in variable --name-- the string value, "--main--" This tells Python, "This specific file is the primary entry point and root of the application."

ii) When ~~a file~~ the same file is imported as a module.

When you import that exact same file into another script or program, Python assign --name-- the actual name of the module.

9) Why do we write the condition:
if __name__ == "__main__":

Ans : We write this condition to control the execution of certain parts of your code. It allows you to write:

- Code that runs only when the file is executed directly
- Code that does not run when the file is imported in another module.

for eg:

```
def Display():
    print("Hellow")
```

10) Explain the difference between:
- return x
- return x, y
- return

Ans:
- returns x hands back a single object. The data type recieved by the caller is exactly whatever type x is
- return x, y hands back a single tuple containing multiple objects. Python functions can only mathematically return one item, the interpreter automatically bundles x & y together.
- return immediately terminates the function and hands back the special single object None.